

# Grundlagen der C-Programmierung

Dr. Henrik Brosenne  
Georg-August Universität Göttingen  
Institut für Informatik

Wintersemester 2024/25

## Ausdrücke und Operatoren

### Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Aufbau von Ausdrücken

**Ausdrücke** setzen sich zusammen aus

- **Operanden** in der Regel Konstanten, Variablen (einschließlich Feldkomponenten) und Funktionsaufrufe,
- **Operatoren**,
- **runden Klammern**.

Bereits verwendete Ausdrücke.

|                          |                                      |
|--------------------------|--------------------------------------|
| <code>i</code>           | als Argument von <code>printf</code> |
| <code>i - 1</code>       | bei der Schleifenzählung             |
| <code>v[i] * w[i]</code> | beim Skalarprodukt                   |
| <code>i &lt; N</code>    | beim Skalarprodukt                   |

Weitere Beispiele für Ausdrücke.

```
(a + b) * 7
(a + b) * (c + d)
- a
```

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

**Wertzuweisung**

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Wertzuweisung

Eine Besonderheit von C ist, dass auch der **Zuweisungsoperator** (=) als Operator betrachtet wird.

$$i = i - 1$$

Das ist formal ein Ausdruck und wird nur durch das nachgestellte Semikolon zu einer Anweisung.

Die Wirkung.

- $i - 1$  wird berechnet,
- der resultierende Wert wird der Variablen  $i$  zugewiesen,
- der resultierende Wert ist gleichzeitig der Wert des gesamten Ausdrucks.

# Wert eines Ausdrucks

Hintereinanderstellen von Ausdrücken.

```
i = (j = (k = 0))
```

Die drei Variablen `i`, `j` und `k` erhalten jeweils den Wert 0.

Mehrere Gleichheitszeichen werden von **rechts nach links** abgearbeitet, deshalb sind die Klammern in diesem Ausdruck überflüssig.

Ein Ausdruck mit gleichem Resultat.

```
i = j = k = 0
```

# Zuweisungsoperator (1/2)

Compiler betrachten den Namen einer Variablen in der Regel als Repräsentanten für den Wert der Variablen.

Deshalb musste bei der Funktion `scanf` den Argumenten den Adressoperator `&` vorangestellt werden, damit die Adresse des Arguments anstelle seines Wertes übergeben wird.

Für die Wertzuweisung gilt entsprechend, nicht der Wert der Variablen, die verändert werden soll, wird benötigt, sondern die Adresse.

Der Standard legt fest, auf der linken Seite des Zuweisungsoperators repräsentiert ein Variablenname die Adresse der Variablen und nicht wie sonst ihren Wert.

Ein (in der Sache unsinnigen) Ausdruck.

```
i = i
```

**Wichtig.** Es stehen `i` links (Adresse) und rechts (Wert) vom Zuweisungsoperator für ganz verschiedene Dinge.

## Zuweisungsoperator (2/2)

Wesentlich wird die Unterscheidung zwischen linker und rechter Seite des Zuweisungsoperators, wenn man sich überlegt, was aus der linken Seite eines Zuweisungsoperators stehen darf und was nicht.

Dass Variablen und Elemente von Feldern erlaubt sind, haben wir bereits gesehen. Nicht erlaubt sind z.B. Konstanten und Ausdrücke

```
5 = i           // nicht zulaessig
i + j = k       // nicht zulaessig
```

Weder die Konstante 5 noch der Ausdruck `i + j` besitzen einen Speicherplatz mit Adresse.

Die Argumente von `scanf` unterliegen denselben Regeln wie die linken Operanden von Wertzuweisungen. Denn das Prinzip ist dasselbe, `scanf` soll den Variablen Werte zuweisen.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

### Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Arithmetische Operatoren

C kennt fünf **arithmetische Operatoren**, nämlich

|   |                                         |
|---|-----------------------------------------|
| + | Addition                                |
| - | Subtraktion                             |
| * | Multiplikation                          |
| / | (ganzzahlige) Division                  |
| % | Modulo (Rest bei ganzzahliger Division) |

Plus (+) und Minus (-) gibt es in zwei Varianten.

- **Unäre** Operatoren dienen als Vorzeichen (positives Vorzeichen erst vom Standard vorgeschrieben).
- **Binäre** Operatoren bewirken Addition bzw. Subtraktion.

# Division (1/2)

Bei der Division ganzzahliger Operanden ist der Quotient wieder ganzzahlig.

Als Ergänzung zum Divisionsoperator / gibt es den **Modulo-Operator %**, der den Rest bei der Division ganzer Zahlen liefert. Dieser Operator darf **nur** für ganzzahlige Operanden verwendet werden, nicht für Gleitkommawerte.

Beispiele für die Division.

|          |        |          |
|----------|--------|----------|
| 10.0 / 3 | ergibt | 3.333333 |
| 10 / 3   | ergibt | 3        |
| 10 % 3   | ergibt | 1        |

## Division (2/2)

Der **Divisionsrest** ist eindeutig bestimmt, wenn beide Operanden nichtnegativ sind; der Divisionsrest ist dann positiv.

Bei mindestens einem negativen Operanden ist es implementationsabhängig, ob der Divisionsrest positiv oder negativ ist.

Die ganzzahlige Division ist auch nur für nichtnegative Operanden eindeutig.

Der Standard schreibt vor, dass folgende Beziehung für beliebige ganzzahlige Werte  $a$  und  $b \neq 0$  erfüllt sein muss.

$$a == (a / b) * b + a \% b$$

Möglich ist zum Beispiel

$$10 / -3 == -3$$

$$10 \% -3 == 1$$

ebenso wie

$$10 / -3 == -4$$

$$10 \% -3 == -2$$

# Auswertung

Arithmetische Operatoren werden prinzipiell von links nach rechts abgearbeitet.

Allerdings gilt es eine **Hierarchie** der Operatoren, durch die die **Präzedenz** (Reihenfolge) der Operatoren festgelegt wird.

Die Operatoren  $*$ ,  $/$  und  $\%$  stehen auf einer Hierachiestufe, die höher ist als die Hierachiestufe auf der  $+$  und  $-$  stehen.

Das ergibt „Punktrechnung vor Strichrechnung“.

Die **unäre** Vorzeichenoperatoren  $(+, -)$  stehen auf einer höheren Hierachiestufe, als die binären Operatoren.

Die Reihenfolge kann durch runde Klammern beeinflusst werden.

## Beispiele

$$a + b * c == a + (b * c)$$

$$a + b - c == (a + b) - c$$

$$- a * b == (- a) * b$$

Beim numerischen Rechnen muss einiges mehr beachtet werden, es wird deshalb separat betrachtet.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

**Inkrementierung und Dekrementierung**

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Inkrementierung und Dekrementierung

C verfügt über spezielle Operatoren zum **Inkrementieren** und **Dekrementieren** von Variablen.

Inkrementierung (++) erhöht den Operanden um 1,  
Dekrementierung (--) verringert den Operanden um 1.

Beide Operatoren können wahlweise in **Präfix**-Schreibweise (vor dem Operanden) oder auch **Postfix**-Schreibweise (hinter dem Operanden) verwendet werden.  
Folgende **Anweisungen** sind äquivalent.

|                   |                   |                         |
|-------------------|-------------------|-------------------------|
| <code>++i;</code> | <code>i++;</code> | <code>i = i + 1;</code> |
| <code>--i;</code> | <code>i--;</code> | <code>i = i - 1;</code> |

# Auswertung

Als **Ausdrücke** sind Präfix- und Postfix-Schreibweise keineswegs äquivalent.

- Bei Präfix-Schreibweise wird **erst** inkrementiert bzw. dekrementiert und **danach** der bereits veränderte Wert der Variablen als Wert des Ausdrucks genommen.
- Bei Postfix-Schreibweise wird **erst** der noch unveränderte Wert der Variablen als Wert des Ausdrucks genommen und **danach** inkrementiert bzw. dekrementiert.

## Beispiele

```
i = 10; j = ++i;
```

i und j haben beide den Wert 11.

```
i = 10; j = i++;
```

i hat den Wert 11, aber j hat den Wert 10.

Interessant sind beide Operatoren vor allem im Zusammenhang mit Schleifen und Vektoren.

# Beispiel (1/2)

```
1  #include <stdio.h>
2
3  #define N 5
4
5  int main() {
6      // declarations -----
7      double v[N], w[N], prod;
8      int i;
9
10     // input -----
11     printf("Enter first vector.\n");        // read 1st vector
12     i = 0;                                  // loop, variation 1
13     while (i < N) {
14         scanf("%lf", &v[i]);
15         i++;
16     }
17     printf("Enter second vector.\n");       // read 2nd vector
18     i = - 1;                                // loop, variation 2
19     while (++i < N) {
20         scanf("%lf", &w[i]);
21     }
22 }
```

## Beispiel (2/2)

---

```
23  // compute scalar product -----
24  prod = 0;
25  i = N;                                // loop, variation 3
26  while (i--) {
27      prod = prod + v[i] * w[i];
28  }
29
30  // output -----
31  printf("Scalar product: %f\n", prod);
32
33  return 0;
34 }
```

---

# Inkrement-Operator

Für die Operanden von Inkrementierung und Dekrementierung gelten dieselben Regeln gelten wie für die Operanden auf der linken Seite eines Zuweisungsoperators.

Es wird eine Adresse benötigt, an der der veränderte Wert gespeichert werden soll.

In der Regel wird man Inkrementierung und Dekrementierung für ganzzahlige Operanden verwenden; erlaubt sind sie aber auch für Gleitkommaoperanden.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

## Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Vergleichsoperatoren

C kennt die sechs **Vergleichsoperatoren**.

|                    |                |
|--------------------|----------------|
| <code>==</code>    | gleich         |
| <code>!=</code>    | ungleich       |
| <code>&lt;</code>  | kleiner        |
| <code>&lt;=</code> | kleiner gleich |
| <code>&gt;</code>  | größer         |
| <code>&gt;=</code> | größer gleich  |

Der Wert eines Ausdrucks mit einem Vergleichsoperator ist gleich 1 (**wahr**), wenn die Operanden in der Relation zueinander stehen, die der Operator beschreibt, sonst gleich 0 (**falsch**).

Letztlich sind die Resultate von Vergleichen `int`-Werte.

Seit C99 steht auch der Datentyp `_Bool` mit den Konstanten `true` und `false` aus `stdbool.h` zur Verfügung.

Z.B. können Ergebnisse von Vergleichsoperationen in `_Bool`-Variablen gespeichert oder mit diesen verglichen werden.

# Vergleichsausdrücke

Vergleichsausdrücke sind `int`-Werte und können deshalb auch Operanden in (arithmetischen) Ausdrücken sein.

Die Vergleichsoperatoren stehen auf einer niedrigeren Hierarchiestufe als die arithmetischen Operatoren, deshalb kommt man in der Regel ohne Klammern aus.

Der Ausdruck

$$a < b + c$$

entspricht also

$$a < (b + c)$$

# == und =

Es kann verhängnisvoll sein, wenn man statt == nur = schreibt.

## Beispiel

```
if (i = 1) {           // immer wahr
    ...
}
```

Hier wird die Anweisungsfolge, die dem `if` folgt, **immer** ausgeführt.

Bei der Auswertung von `i = 1` erhält die Variable `i` den Wert 1. Auch der Wert des Ausdrucks insgesamt ist damit 1, wird entsprechend als **wahr** interpretiert und bewirkt die Ausführung der unmittelbar folgenden Anweisungen.

Auch Endlosschleifen kann man so bequem (und ungewollt) „zusammenbasteln“.

```
while (i = 1) {       // Endlosschleife
    ...
}
```

Änderungen von `i` im Schleifenrumpf haben hierauf keinen Einfluss, da der Ausdruck `i = 1` vor jedem Schleifendurchgang erneut ausgewertet wird.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

**Logische Operatoren**

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Logische Operatoren

C kennt drei **logische Operatoren**.

|                         |       |
|-------------------------|-------|
| <code>&amp;&amp;</code> | und   |
| <code>  </code>         | oder  |
| <code>!</code>          | nicht |

Anwenden darf man diese Operatoren auf beliebige Operanden, die sich mit 0 vergleichen lassen, das sind z.B. Ausdrücke mit einem Ganzzahl-/Gleitkommazahltyp oder Ausdrücke vom Typ `_Bool`.

Es wird nach der bereits bekannten Regel vorgegangen, der Wert 0 wird als **falsch**, alle anderen Werte werden als **wahr** interpretiert.

Der Wert eines Ausdrucks mit logischem Operator ist entweder 0 oder 1.

# Wahrheitstafeln für logische Operatoren

| a        | ! a |
|----------|-----|
| 0        | 1   |
| $\neq 0$ | 0   |

| a        | b        | a && b |
|----------|----------|--------|
| 0        | 0        | 0      |
| $\neq 0$ | 0        | 0      |
| 0        | $\neq 0$ | 0      |
| $\neq 0$ | $\neq 0$ | 1      |

| a        | b        | a    b |
|----------|----------|--------|
| 0        | 0        | 0      |
| $\neq 0$ | 0        | 1      |
| 0        | $\neq 0$ | 1      |
| $\neq 0$ | $\neq 0$ | 1      |

# Auswertung

Logische Ausdrücke werden von **links nach rechts** abgearbeitet, wobei die Hierachiestufe der Operatoren berücksichtigt wird.

- erst **nicht**,
- dann **und**,
- zuletzt **oder**.

**Besonderheit.** Der Standard legt fest, dass die Auswertung eines logischen Ausdrucks beendet wird, sobald das Ergebnis feststeht.

Diese Besonderheit ist ausgesprochen nützlich.

## Beispiel

```
(d != 0) && (c / d < 10)
```

Die Division wird **nicht** ausgeführt, wenn d gleich 0 ist.

Der erste Vergleich stellt fest, dass das Resultat **falsch** ist – unabhängig vom Wert des anderen Operanden – und verhindert so die dann nicht zulässige Division.

Man von spricht von *fauler Auswertung* (engl. **lazy evaluation**).

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

## Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Zusammengesetzte Zuweisungen

Neben dem einfachen Zuweisungsoperator `=` gibt es in C zehn **zusammengesetzte Zuweisungsoperatoren**.

Zusammengesetzte Zuweisungsoperatoren haben einheitlich die Form `op=`, wobei `op` ein binärer Operator ist.

Der Standard schreibt vor, dass zwischen dem binären Operator und dem Gleichheitszeichen **kein** Leerzeichen stehen darf.

Sei `var` eine Variable und `ausdr` eine Ausdruck.

`var op= ausdr` entspricht `var = var op ausdr`

## Auswertung

- Der Ausdruck `ausdr` wird ausgewertet.
- Der Wert von `ausdr` wird mit dem Wert der Variablen `var` durch den Operator `op` verknüpft.
- Das Resultat der Verknüpfung wird in der Variablen `var` gespeichert.
- Der gespeicherte Wert ist der Wert des gesamten Ausdrucks.

# Erlaubte Operatoren

Nicht alle binären Operatoren sind bei der Zusammensetzung erlaubt.

Von den bis jetzt eingeführten Operatoren, dürfen nur die fünf arithmetischen Operatoren verwendet werden.

## Beispiele

|                              |            |                                          |
|------------------------------|------------|------------------------------------------|
| <code>x += 3.5</code>        | entspricht | <code>x = x + 3.5</code>                 |
| <code>i *= j + 10</code>     | entspricht | <code>i = i * (j + 10)</code>            |
| <code>a += b += c</code>     | entspricht | <code>a = a + (b = b + c)</code>         |
| <code>v[i + j*k] += 3</code> | entspricht | <code>v[i + j*k] = v[i + j*k] + 3</code> |

Zusammengesetzten Zuweisungsoperatoren können Schreibarbeit sparen helfen.

Dem Compiler wird es erleichtert, effizienten Code zu erzeugen.

Der Indexausdruck im letzten Beispiel braucht nur einmal berechnet zu werden. Verwendet man die zusammengesetzte Zuweisung, so steht der Indexausdruck auch nur einmal im Programm, so dass der Compiler ihn nicht doppelt berechnen muss.

Verwendet man die einfache Zuweisung, muss man darauf hoffen, dass der Compiler die Identität erkennt und die Doppelberechnung wegoptimiert.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

**Der Kommaoperator**

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Der Kommaoperator (1/2)

Der **Kommaoperator** fasst mehrere Ausdrücke zu einem einzigen Ausdruck zusammen.

Beispiel Die Werte der beiden Variablen **x** und **y** sollen vertauscht werden.

```
h = x;  
x = y;  
y = h;
```

Man kann auch schreiben.

```
h = x, x = y, y = h;
```

Dass dieses kürzer scheint liegt ausschließlich daran, dass jeweils eine Anweisung in eine Zeile geschrieben wurde, was der Standard keineswegs verlangt.

Durch Kommaoperatoren getrennte Ausdrücke werden von **links nach rechts** ausgewertet.

Der Wert des Gesamtausdrucks ist der Wert des am weitesten rechts stehenden, zuletzt ausgewerteten Teilausdrucks.

## Kommaoperator (2/2)

Der Kommaoperator sollte in einer Anweisung nur dann verwendet werden, wenn Konstruktionen sehr eng zusammengehören.

Beispiel. Den Tausch der Werte von zwei Variablen kann man durchaus als eine Operation ansehen.

An anderer Stelle wird sich der Kommaoperator als sehr nützlich erweisen.

Kommata, die in Variablenvereinbarungen und Funktionsaufrufen stehen, haben **nichts** mit dem Kommaoperator zu tun. Entsprechend haben sie dort auch keinen Einfluss auf die Reihenfolge der Auswertung.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

### **Nebeneffekte**

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Nebeneffekte

Unter einem **Nebeneffekt** versteht man bei der Programmierung, dass als Nebenprodukt der **Auswertung eines Ausdrucks** der Wert einer Variablen verändert wird.

C die *Sprache der Nebeneffekte*.

Die Wertzuweisung ist keine Anweisung, sondern ein Operator, deshalb lassen sich die Werte von Variablen **nur** durch Nebeneffekte verändern.

Neben den Zuweisungsoperatoren bewirken auch die Inkrement- und Dekrementoperatoren sowie alle Funktionen mit Ausgabeparametern Nebeneffekte.

Beispiel

```
while (scanf (...) == 1)
    ...
```

# Nebeneffekte

Der Standard schreibt nur sehr eingeschränkt vor, zu welchem Zeitpunkt Nebeneffekte wirksam werden müssen.

- Nebeneffekte können natürlich nicht eintreten, bevor die Auswertung des Ausdrucks beginnt, durch die sie hervorgerufen werden.
- Wenn die Auswertung eines Ausdrucks abgeschlossen ist, müssen auch seine Nebeneffekte eingetreten sein.

Folge. Wenn eine Variable innerhalb eines Ausdrucks mehrfach angesprochen wird und außerdem Nebeneffekte für sie eintreten, ist meist **nicht** definiert, welchen Wert die Variable im Einzelfall besitzt.

## Beispiele

Folgende Anweisungen können implementationsspezifisch unterschiedliche Resultate haben.

```
v[i] = i++;  
v[i] = w[i++];  
s += v[i] * w[i++];  
b = a++ + a;
```

# Probleme (1/3)

Probleme können sich aus der Tatsache ergeben, dass die Reihenfolge der Auswertung von Ausdrücken nur teilweise festgelegt ist.

- Ausdrücke, die mit dem Kommaoperator verknüpft sind, werden von links nach rechts ausgewertet.  
Dabei treten alle Nebeneffekte auf, bevor der rechte Ausdruck ausgewertet wird.
- Es ist zwar festgelegt, in welcher Reihenfolge aufeinanderfolgende Operatoren gleicher Hierarchiestufe ausgeführt werden, nicht aber die Reihenfolge der Berechnung der Operanden.
- Logische Ausdrücke werden von links nach rechts ausgewertet; ihre Berechnung wird beendet, sobald das Resultat feststeht.

Bei `||` ist das der Fall, sobald der erste Teilausdruck wahr ist, bei `&&` sobald der erste Teilausdruck falsch ist.

Nebeneffekte treten jeweils auf, bevor mit dem nächsten Teilausdruck fortgefahren wird.

# Probleme, Reihenfolge (1/2)

## Beispiel

Reihenfolge der Berechnung der Operanden ist nicht festgelegt.

$$(a) + (b) + (c)$$

wird ausgewertet wie

$$((a) + (b)) + (c)$$

Sind  $a$ ,  $b$  und  $c$  ihrerseits Ausdrücke, so ist **nicht** festgelegt, welcher von ihnen zuerst berechnet wird.

Folgende Anweisungsfolge kann in der Variablen  $j$  je nach Implementation den Wert 14 oder 17 liefern.

```
i = 5;  
j = i * 3 + ++i - 7;
```

# Probleme, Reihenfolge (2/2)

Ebenfalls ist nicht festgelegt, in welcher Reihenfolge die Argumente eines Funktionsaufrufs berechnet werden.

Folgende Anweisungsfolge kann je nach Implementation die Ausgabe 4, 6 oder 4, 8 ergeben.

```
n = 3;  
printf("%d, %d\n", ++n, 2*n);
```

# Probleme, lazy evaluation

Die scheinbar problemlosen Operatoren bieten „Fallstricke“, weil bei ihnen Teilausdrücke unter Umständen nicht ausgewertet werden und in ihnen steckende Nebeneffekte entsprechend mal eintreten und mal nicht

## Beispiel

```
if (x == y && a == ++b)           // 1
    ...
if (x == y || a == ++b)           // 2
    ...
z = x == y ? a : ++b;             // 3
```

Im ersten Beispiel wird **b** nur dann inkrementiert, wenn **x == y** gilt.

Im zweiten und dritten Beispiel wird **b** nur dann inkrementiert, wenn **x != y** gilt.

**Fazit.** Bei Operatoren und Funktionen, die Nebeneffekte erzeugen, sollte man **äußerste** Vorsicht walten lassen – zumal man Fehler im Programm, die aus Nebeneffekten resultieren, oft nur mit großer Mühe findet. Insbesondere sollte man darauf achten, dass Variablen, die inkrementiert oder dekrementiert werden, **innerhalb eines Ausdrucks** nur an der Stelle vorkommen, an der sie inkrementiert bzw. dekrementiert werden.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

## Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Typumwandlung

**Frage.** Welche Typen dürfen bzw. müssen die Operanden eines binären arithmetischen Operators besitzen?

**Antwort.** Jede beliebige Kombination ist erlaubt.

Beispiel

$$10.0 / 3 = 3.333333$$

Welchen Wert erhalten solche Ausdrücke?

Der Rechner kann grundsätzlich nur Operanden mit identischen Typen verknüpfen, so dass also vor die eigentliche Verknüpfung ggf. eine **Typumwandlung** vorgeschaltet werden muss.

Die Typumwandlung übernimmt in der Regel der C-Compiler in der Regel für Programmierer, so eine Typumwandlung wird **implizite Typumwandlungen** bezeichnet.

Das Resultat einer Verknüpfung hat den Typ der beteiligten (umgewandelten) Operanden, man spricht deshalb auch vom **Typ eines Ausdrucks**.

# Typumwandlung

## Beispiel

Verknüpfung von zwei Operanden durch einen binären arithmetischen Operator.

- Mit Werten der Typen `char` und `short`, mit oder ohne Vorzeichen, wird nicht gerechnet, d.h. Werte mit diese Typen werden vor dem Rechnen auf jeden Fall automatisch in `int` bzw. `unsigned int` umgewandelt.

Ein Ausdruck wie `'Z' - 'A'` hat also den Typ `int`.

- Bei Operanden mit unterschiedlichen Typen wird der Operand mit dem **niederwertigen** Typ in den **höherwertigen** Typ umgewandelt.

# Hierarchie der impliziten Typumwandlung

## long und int ungleich

---

9. long double
8. double
7. float
6. unsigned long long
5. long long
4. unsigned long
3. long
2. unsigned int
1. int

## long und int gleich

---

7. long double
6. double
5. float
4. unsigned long long
3. long long
2. unsigned int/long
1. int/long

Die Typen `char` und `short` kommen nicht vor, weil diese Typen immer nach `int` umgewandelt werden.

# Entwicklung des Typs

Bei komplexeren Ausdrücken entwickelt sich der Typ im Laufe der Auswertung. Insbesondere wird nicht zuerst der Typ des Resultats bestimmt und erst danach die Berechnung ausgeführt. Dahinter steht, dass für die Auswertung jedes Teilausdrucks der einfachste möglichen Typ verwendet werden soll.

# Beispiel

## Beispiel

```
short  s = 65;
int    i = 7;
long   j = 1000;
float  x = 3.0;
double y = 46.5;

printf("%f\n", s * i + j % i + y / x);
```

Die angegebene Reihenfolge muss nicht unbedingt eingehalten werden.

- Ausdruck `s * i` wird `s` in `int` umgewandelt.  
Multiplikation liefert `int`-Wert 455.
- Ausdruck `j % i` wird `i` in `long` umgewandelt.  
Divisionsrest ist `long`-Wert 6.
- Ausdruck `y / x` wird `x` in `double` umgewandelt.  
Quotient ist `double`-Wert 15.5.

Die Additionen **müssen** nun von links nach rechts ausgeführt werden:

- `int`-Wert 455 wird in `long` umgewandelt und zum `long`-Wert 6 addiert.  
Die Summe ist der `long`-Wert 461.
- `long`-Wert 461 wird in `double` umgewandelt und zum `double`-Wert 15.5 addiert.  
Summe/Gesamtresultat ist `double`-Wert 476.5.

# Fallstrick ganzzahlige Divisionen

Ein häufig gemachter Fehler ist die unbeabsichtigte ganzzahlige Divisionen.

So liefert der Ausdruck `1 / 2 * 3.5` **stets** den Wert 0.

In derart offensichtlicher Form kommt dieser Fehler in der Praxis natürlich selten vor. Häufiger ist er mit zwei `int`-Variablen, etwa `i / j * 3.5` – was an der ganzzahligen Division nichts ändert.

# Fallstrick Zuweisungsoperator (1/2)

Die Zuweisungsoperatoren nehmen eine Sonderstellung ein.

Es ist einer Variablen ein Wert zuzuweisen – der Typ einer Variablen kann aber während der Ausführung eines Programms nicht verändert werden kann.

Typumwandlung im Zuge einer Wertzuweisung ist also zwar ggf. erforderlich, kann aber stets nur in einer Umwandlung des Typs des Wertes in den Typ der Variablen bestehen.

# Fallstrick Zuweisungsoperator (2/2)

Es kann erforderlich sein, dass aus einem höherwertigen in einen niederwertigen Typ umgewandelt wird.

- Bei der Umwandlung zwischen ganzzahligen Typen werden nur die wertniedrigsten Bits übernommen.
- Bei der Umwandlung eines Gleitkommawertes in einen ganzzahligen Typ gehen eventuelle Nachkommastellen verloren. Eine Rundung erfolgt bei dieser Umwandlung **nicht**! So erhält die `int`-Variable `i` durch die Wertzuweisung `i = 3.9;` den Wert 3!
- Wenn bei Gleitkommatypen aus höherer in niedrigere Genauigkeit umgewandelt wird (z.B. von `double` in `float`), bleibt es den Implementatoren überlassen, ob dabei gerundet oder abgeschnitten wird. (Das ist bei binären Darstellungen letztlich gleichwertig!)
- Bei der Umwandlung aus einem „großen“ ganzzahligen Typ in einen „kleinen“ Gleitkommatyp kann Genauigkeit verlorengehen.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

**Castoperatoren**

Der `sizeof`-Operator

Hierarchie der Operatoren

# Castoperatoren

Gelegentlich ist es wünschenswert, den Compiler dazu zwingen zu können, einen Wert in einen bestimmten Typ umzuwandeln, etwa zur Vermeidung von unerwünschten ganzzahligen Divisionen.

Hierfür stellt C die **Castoperatoren** zur Verfügung.

Ein Castoperator ist eine Typbezeichnung, die in runde Klammern eingeschlossen wird. Castoperatoren sind immer unär und haben sehr hohe Priorität.

## Beispiel

Berechnung des `double`-Quotient von zwei `int`-Variablen `i` und `j`.

```
(double) i / j  
i / (double) j
```

Der Castoperator wirkt dabei wegen seiner hohen Priorität jeweils nur auf einen der beiden Operanden. Der andere Operand wird dann gemäß der Regeln der impliziten Typumwandlung ebenfalls nach `double` umgewandelt.

Alle Probleme der Umwandlungen, die durch Wertzuweisungen erzwungen werden, können offensichtlich auch bei der Verwendung von Castoperatoren auftreten.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der **sizeof**-Operator

Hierarchie der Operatoren

# sizeof-Operator (1/2)

Mit dem `sizeof`-Operator lässt sich der Speicherbedarf für Datentypen ermitteln. Das wird später eine wichtige Rolle spielen, wenn dynamisch Speicher bereitgestellt werden soll.

```
sizeof ausdruck  
sizeof (typ)
```

Im ersten Fall ist das Resultat der Speicherbedarf der benötigt würde, um eine einfache Variable vom Typ des Ausdrucks zu speichern.

Im zweiten Fall ist das Resultat der Speicherbedarf einer einfachen Variablen mit dem angegebenen Typ.

## sizeof-Operator (2/2)

Die Maßeinheit ist rechner-spezifisch, wird jedoch dadurch festgelegt, dass der Standard `sizeof (char) == 1` vorschreibt.

Festgelegt ist außerdem, dass das Resultat den Typ `size_t` besitzt – oder umgekehrt.

Der Typ `size_t` ist so zu definieren, dass sein Wertebereich alle möglichen Resultate des Operators `sizeof` umfasst.

`size_t` ist demnach kein elementarer Datentyp, sondern wird in den Headerdateien `stddef.h`, `stdlib.h` und `string.h` deklariert.

# Zu Beachten (1/2)

Bei der Verwendung des `sizeof`-Operators sind einige Dinge besonders zu beachten.

Handelt es sich bei *ausdruck* um den Namen eines Feldes, bzw. bei *typ* um einen Feldtyp, so wird sowohl die Dimension des Feldes als auch die Größe der einzelnen Elemente berücksichtigt.

```
int feld[7];  
size_t s;  
s = sizeof (feld);           // s == 7 * sizeof(int)  
s = sizeof (char [2][5]);    // s == 10
```

## Zu Beachten (2/2)

Der `sizeof`-Operator ist trotz seiner Form nicht mit einem Funktionsaufruf wie z.B. bei `printf(...)` zu verwechseln.

Ganz speziell gilt, dass keine Nebeneffekte für den Ausdruck *ausdruck* auftreten. Der Compiler ermittelt lediglich den **Typ** des Ausdrucks.

Zur Laufzeit des Programms werden keine weiteren Operationen ausgeführt. Das kann zu einiger Verwirrung führen:

```
int s = 0;
printf("%ld\n", sizeof (5 / s)); // sizeof (int)
printf("%ld\n", sizeof (s = 5)); // sizeof (int)
printf("%d\n", s);               // 0
```

Es sind also sogar Ausdrücke erlaubt, die zur Laufzeit einen Fehler verursachen würden (im Beispiel Division mit 0). Da dieser Ausdruck jedoch formal den Typ `int` besitzt, wird als Wert `sizeof(int)` ermittelt.

## Ausdrücke und Operatoren

Aufbau von Ausdrücken

Wertzuweisung

Arithmetische Operatoren

Inkrementierung und Dekrementierung

Vergleichsoperatoren

Logische Operatoren

Zusammengesetzte Zuweisungen

Der Kommaoperator

Nebeneffekte

Typumwandlung

Castoperatoren

Der `sizeof`-Operator

Hierarchie der Operatoren

# Hierarchie der Operatoren

| Stufe | Operator      | Beschreibung                                                             | Auswertungsreihenfolge |
|-------|---------------|--------------------------------------------------------------------------|------------------------|
| 15    | ( )           | Auswertung von Parametern                                                | →                      |
|       | [ ]           | Auswahl einer Feldkomponente                                             |                        |
|       | -> .          | Auswahl einer Strukturkomponente                                         |                        |
| 14    | ! ~           | Negation (logisch, bitweise)                                             | ←                      |
|       | ++ -          | Inkrementierung, Dekrementierung<br>(Präfix oder Postfix)                |                        |
|       | + -           | Vorzeichen (unär)                                                        |                        |
|       | (Typ)         | Typumwandlung                                                            |                        |
|       | & *<br>sizeof | Adressbildung, Dereferenzierung (unär)<br>Bestimmung des Speicherbedarfs |                        |
| 13    | * /           | Multiplikation (binär), Division                                         | →                      |
|       | %             | Rest bei ganzzahliger Division                                           |                        |
| 12    | + -           | Summe, Differenz (binär)                                                 | →                      |
| 11    | « »           | bitweise Verschiebung nach links, rechts                                 | →                      |
| 10    | < <=          | Vergleich auf kleiner, kleiner oder gleich                               | →                      |
|       | > >=          | Vergleich auf größer, größer oder gleich                                 |                        |
| 9     | == !=         | Vergleich auf gleich, ungleich                                           | →                      |
| 8     | &             | Und (bitweise)                                                           | →                      |
| 7     | ^             | exklusives Oder (bitweise)                                               | →                      |
| 6     |               | inklusives Oder (bitweise)                                               | →                      |
| 5     | &&            | Und (logisch)                                                            | →                      |
| 4     |               | inklusives Oder (logisch)                                                | →                      |
| 3     | ? :           | bedingte Auswertung (nur paarweise!)                                     | ←                      |
| 2     | =             | Wertzuweisung                                                            | ←                      |
|       | +=            | zusammengesetzte Wertzuweisung<br>(auch: -=, *=, /=, <=, >=, &=, ^=,  =) |                        |
| 1     | ,             | sequentielle Auswertung                                                  | →                      |

# Beispiele

Allzu extensive Nutzung der Kenntnis der Hierarchie lohnt sich nicht. Leicht können völlig unleserliche Ausdrücke resultieren.

## Beispiele

```
i - j           // 1: binaer
i -- j          // 2: unzulaessig
i - - j         // 3: binaer/unaer
i --- j         // 4: Dekrement/binaer (wie 6.)
i - - - j       // 5: binaer/Dekrement
i -- - j        // 6: Dekrement/binaer
i ---- j        // 7: unzulaessig
i - --- j       // 8: unzulaessig
i -- -- j       // 9: unzulaessig
i --- - j       // 10: Dekrement/binaer/unaer
i ----- j      // 11: unzulaessig
i - ----- j   // 12: unzulaessig
i -- --- j      // 13: unzulaessig
i --- -- j      // 14: Dekrement/binaer/Dekrement
i ---- - j      // 15: unzulaessig
i ----- j      // 16: unzulaessig
```

Falls ein Ausdruck dieser Art benötigt wird, erhöht es die Lesbarkeit den Ausdruck zu klammern, auch wenn es nicht zwingend notwendig ist.